

LLD Case Study 6: Distributed Inventory Management System

Target Patterns: Observer Pattern, Strategy Pattern, State Pattern

Key Engineering Focus: Stock level invariants, multi-warehouse fulfillment strategies, automatic replenishment alerts, and order lifecycle states.

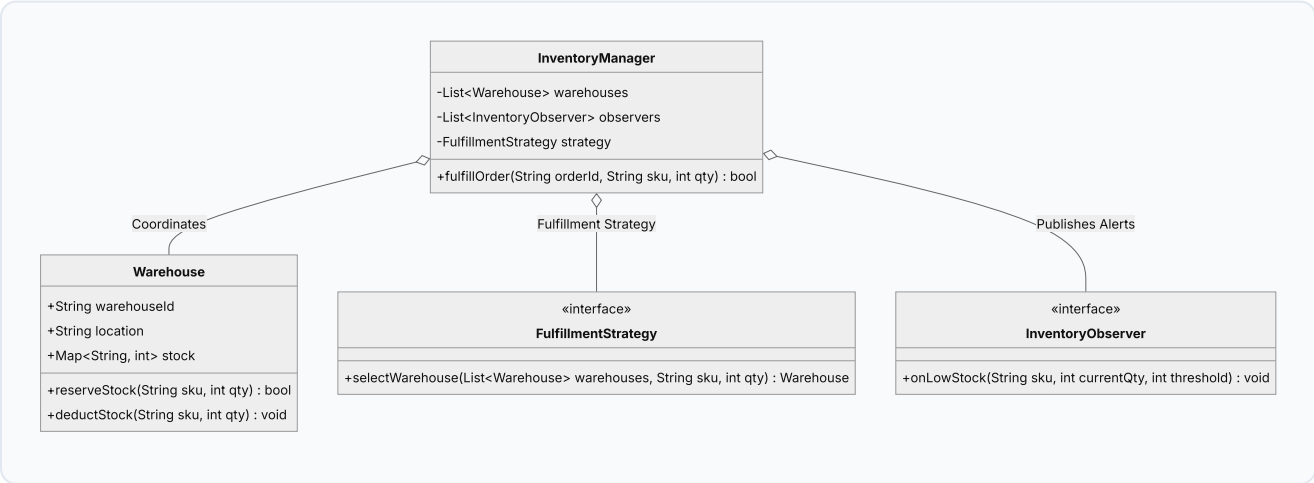
1. Problem Statement & Functional Requirements

Design an enterprise inventory management backend supporting multi-warehouse fulfillment and low-stock alerts.

Requirements:

- 1. Multi-Warehouse Fulfillment (Strategy Pattern):** Route orders to warehouses based on customizable strategies (e.g. Nearest Warehouse First vs Consolidate into Single Shipment).
- 2. Low Stock Alerts (Observer Pattern):** When stock for a SKU drops below its threshold, automatically trigger replenishment purchase orders.
- 3. Order Inventory State Machine (State Pattern):** Orders transition from `Draft` → `StockReserved` → `Dispatched` → `Delivered` (or `Released` if canceled).
- 4. Thread-Safe Inventory Mutations:** Prevent overselling under high concurrency.

2. Architecture & UML Class Diagram



3. Production-Grade Python Implementation

4. Edge Cases, Tests and Extensions

Where the design is right and where it leaks

Concern	Status	Detail
Overselling under concurrency	Safe	<code>deduct</code> checks and subtracts inside one lock, so two threads can never both take the last unit
Pick-then-deduct race	Weak	<code>pick_warehouse</code> reads stock without holding the lock; the winner can be drained before <code>deduct</code> runs, and <code>fulfill_order</code> then fails the whole order even though another warehouse could serve it
Low-stock alert	Fires after every deduction at or below the threshold	Alert storms: fire once per crossing, then re-arm when stock rises above the threshold
Multi-line orders	Not modelled	Reserve all lines or none; otherwise partial fulfilment strands stock
Reservations vs stock	Not modelled	Carts need a time-limited reservation, separate from on-hand quantity

Tests, including the race

The first test proves there is no overselling. The second reproduces the pick-then-deduct race deterministically, and shows the fix: try candidates in preference order until one deduction succeeds. This block extends the implementation above.

Extensions interviewers ask for

1. **Reservations:** add `reserve(sku, qty, ttl)` that moves units from `available` to `reserved`; a background sweep returns expired reservations. Checkout converts a reservation into a

deduction.

2. **Atomic multi-warehouse split orders:** if no single warehouse has 5 units, take 3 from one and 2 from another; do it as a two-phase reserve-then-commit so a failure rolls everything back.
3. **Database-backed stock:** replace the in-process lock with `UPDATE stock SET qty = qty - :n WHERE sku = :s AND qty ≥ :n` and check the affected row count; the database performs the same check-and-subtract atomically.
4. **Cheapest shipping instead of nearest:** swap the `FulfillmentStrategy`; nothing else changes, which is the point of the pattern.

Follow-up questions

- *Why is a lock per warehouse better than one global lock?* Orders for different warehouses never contend; the cost is that cross-warehouse operations need a consistent lock order to avoid deadlock.
- *How do you stop alert storms?* Keep a per-SKU `alerted` flag, set it on the downward crossing, clear it when stock goes back above the threshold.
- *What changes when stock lives in a database?* The check-and-subtract moves into one SQL statement or a `SELECT ... FOR UPDATE` transaction; the in-memory lock disappears and idempotency keys on orders become the new concern.